

CG2111A Engineering Principles and Practice II

Studio 4: Pulse Width Modulation (PWM) Programming

Answer Book

Choose ONE person to submit this report in PDF format, and name the report using the submitter's student number. For example, if the submitter's student number is A1495832X, then the PDF file should be called A1495832X.pdf.

Please submit ONE REPORT per sub-team.

Student Information

Submitting Student Name: Leon Matthew Wei

Submitting Student ID: A0320416U

Student 2 Name: Li Yi An

Student 2 ID: A0334942B

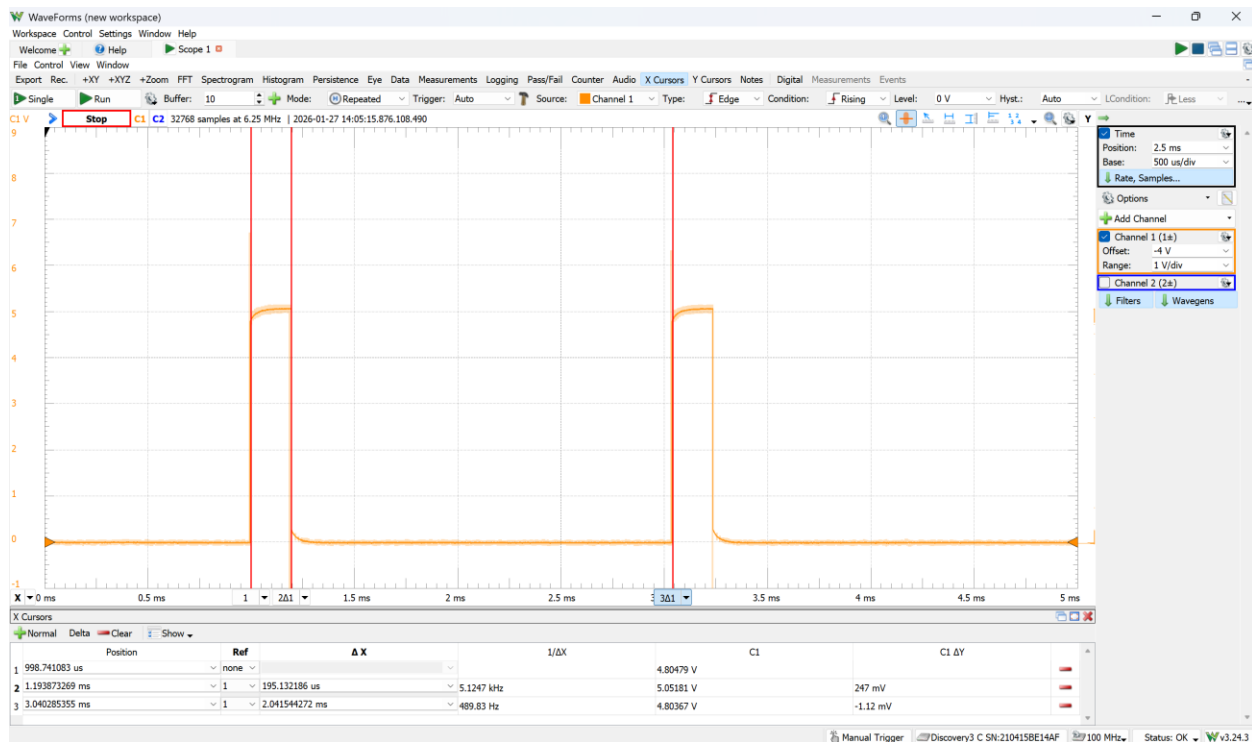
Answers

Please answer all questions below:

Q1: [3 marks]

PWM Waveform Analysis Take a screenshot of the waveform in your software and include it in your report. You must take measurements on your waveform as the format shown in Section 1.3. a) Measure the period of your waveform b) Measure the pulse width (ON time) of your waveform Note down the following parameters in 2 decimal places from the waveform: (a) Period (b) Pulse Width;

Answer:



$$\text{Period of waveform (s)} = 2.04 \times 10^{-3}(\text{s})$$

$$\text{Pulse width, ON Time (s)} = 1.95 \times 10^{-4}(\text{s})$$

Q2: [1 marks]

Calculate the Duty Cycle based on the formula given earlier.

Answer:

$$\text{Duty Cycle (\%)} = \frac{\text{ON Time (s)}}{\text{Period (s)}} \times 100\%$$

$$\text{Duty Cycle (\%)} = \frac{1.9513 \times 10^{-4}}{2.0415 \times 10^{-3}} \times 100\% = 9.56 (\%)$$

Q3: [1 marks]

Explain: why don't we need to write an ISR to generate a PWM signal?

Answer:

The PWM signal is generated by a timer, which is independent from interrupts thus ISR is not needed.

Q4: [3 marks]

Program a complement waveform You are now tasked to generate a complement of the current waveform (HIGH becomes LOW and LOW becomes HIGH). Provide the code change that is needed to do this **WITHOUT** changing the OCR0A register value. State the changes that you made in the code and provide an explanation on how your changes achieve the desired outcome.

Answer:

```
TCCR0A = 0b11000001; // Set OCOM0A to 11 and WGM to 01
```

Explanation:

Bit	7	6	5	4	3	2	1	0
	COM0A1	COM0A0	COM0B1	COM0B0			WGM01	WGM00
Access	R/W	R/W	R/W	R/W			R/W	R/W
Reset	0	0	0	0			0	0

Figure 8: Waveform Generation Mode and Compare Output Mode bit descriptions

COM0A1	COM0A0	Description
0	0	Normal port operation, OC0A disconnected.
0	1	WGM02 = 0: Normal Port Operation, OC0A Disconnected. WGM02 = 1: Toggle OC0A on Compare Match.
1	0	Clear OC0A on Compare Match when up-counting. Set OC0A on Compare Match when down-counting.
1	1	Set OC0A on Compare Match when up-counting. Clear OC0A on Compare Match when down-counting.

Figure 10: Compare Output Mode, Phase Correct PWM Mode

Current configuration: COM0A1 = 1, COM0A0 = 0

Current output: When TCNT0 counts up from 0 to OCR0A, OC0A is **cleared** (set to LOW)

When TCNT0 counts down from 255 to OCR0A, OC0A is **set** (set to HIGH)

New configuration: COM0A1 = 1, COM0A0 = 1

New output: When TCNT0 counts up from 0 to OCR0A, OC0A is **set** (set to HIGH)

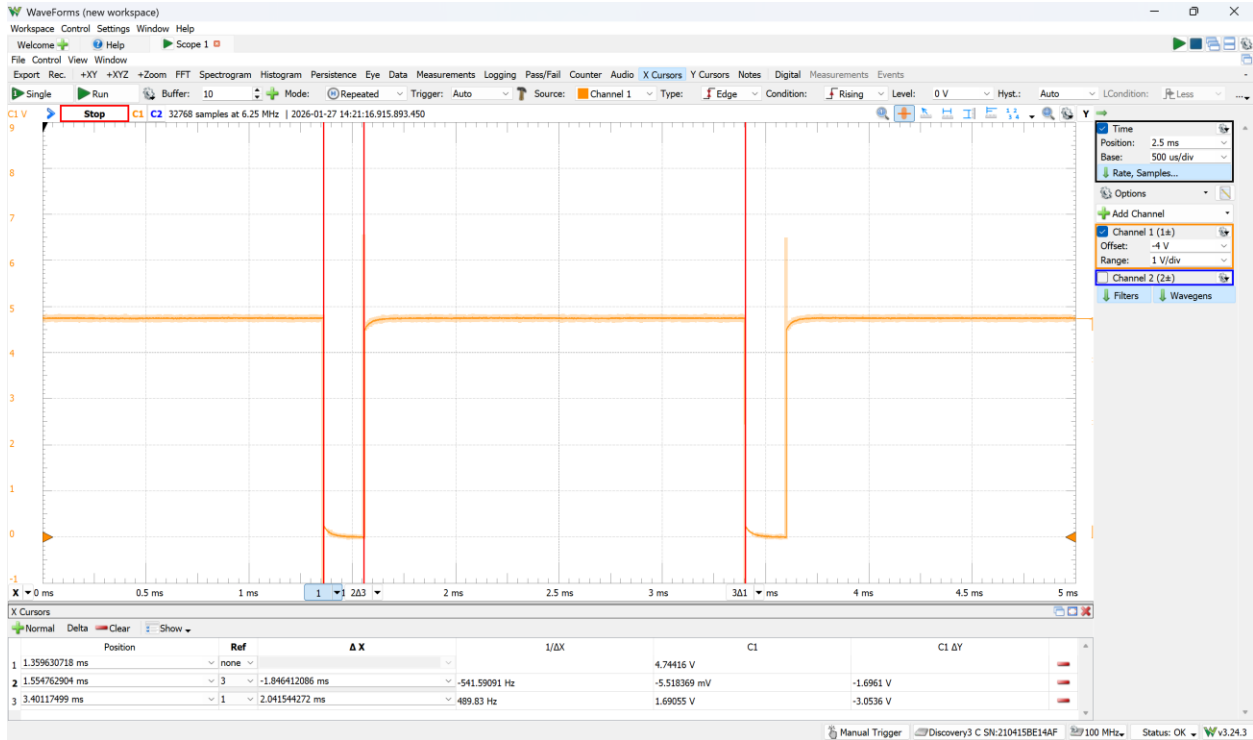
When TCNT0 counts down from 255 to OCR0A, OC0A is **cleared** (set to LOW)

Therefore, instead of **TCCR0A = 0b10000001**, use **TCCR0A = 0b11000001**

Q5: [2 marks]

Screenshot and include a copy of the waveform in your report. You must take measurements on your waveform as the format shown in Section 1.3.

Answer:



$$\text{Period of waveform (s)} = 2.04 \times 10^{-3}(\text{s})$$

$$\text{Pulse width, ON Time (s)} = 1.85 \times 10^{-3}(\text{s})$$

$$\text{Duty Cycle (\%)} = \frac{1.8464 \times 10^{-3}}{2.0415 \times 10^{-3}} \times 100\% = 90.44 (\%)$$

Q6: [3 marks]

Fade In / Fade-Out Effect Implement a Fade In / Fade Out effect for the LED. That is, it should gradually become dimmer until it is unlit, and then gradually become brighter until it is at full brightness, and then repeat the process continually. You must ONLY change the code in the ISR. Other parts of the code must NOT be changed, but you are free to declare additional variables in your code. Include your ISR code in the report and state the declaration of any additional variables.

Answer:

```
#define BRIGHTNESS_FACTOR 5 // Varies the brightness range of LED
#define FLASH_FACTOR 2 // Varies the speed of flashing cycle of LED
static volatile bool counting_up = false; // Variable that toggles between modes
```

```
ISR(TIMER0_COMPA_vect) {
    if (OCR0A <= BRIGHTNESS_FACTOR) {
        counting_up = true;
    } else if (OCR0A >= 255 - BRIGHTNESS_FACTOR) {
        counting_up = false;
    }

    if (counting_up) {
        OCR0A += FLASH_FACTOR;
    } else {
        OCR0A -= FLASH_FACTOR;
    }
}
```

Q7: [2 marks]

This fading effect is very fast, so let's slow it down a little. Reduce the speed of the fading effect. Provide your code snippet for this in your report. Hint: you only need to change the value in one register for this effect.

Answer:

Changes made: FLASH_FACTOR changed from 2 to 1

```
#define BRIGHTNESS_FACTOR 5 // Varies the brightness range of LED
#define FLASH_FACTOR 1 // Varies the speed of flashing cycle of LED
static volatile bool counting_up = false; // Variable that toggles between modes
```

```
ISR(TIMER0_COMPA_vect) {
    if (OCR0A <= BRIGHTNESS_FACTOR) {
        counting_up = true;
    } else if (OCR0A >= 255 - BRIGHTNESS_FACTOR) {
        counting_up = false;
    }

    if (counting_up) {
        OCR0A += FLASH_FACTOR;
    } else {
        OCR0A -= FLASH_FACTOR;
    }
}
```

Q8: [3 marks]

Which Timer? Explain why an 8-bit Timer/Counter cannot generate a 50 Hz PWM signal. In your answer, calculate the closest achievable frequencies using the 8-bit Timer/Counter and show that they deviate significantly from 50 Hz, either being too high or too low.

Answer:

$$f_{OCnxPCPWM} = \frac{16 \times 10^6}{N \times 2 \times 255} = 50Hz$$

$$N = 627.45$$

Thus, the closest available N values are either N = 256 or N = 1024

$$\text{However, when } N = 256, f_{OCnxPCPWM} = \frac{16 \times 10^6}{256 \times 2 \times 255} = 122.55Hz$$

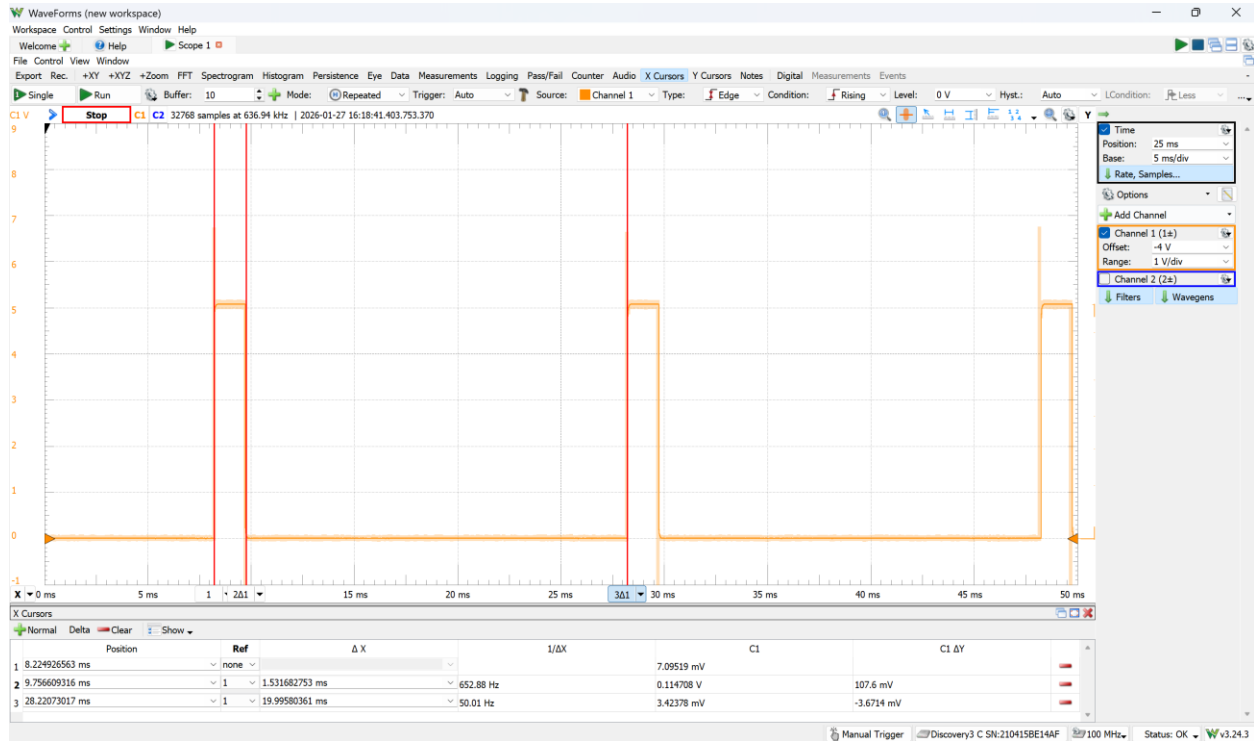
$$\text{and when } N = 1024, f_{OCnxPCPWM} = \frac{16 \times 10^6}{1024 \times 2 \times 255} = 30.63Hz$$

Since 122.55Hz and 30.63Hz are too different from the desired frequency of 50Hz, an 8-bit Timer is not suitable.

Q9: [5 marks]

Use a 16-bit timer (Timer 1) to generate a 50 Hz waveform. Assign the appropriate values to the registers in the setup() function. Refer to the datasheet to determine the correct register values. a) Attach the waveform screenshot in your lab report. b) Paste your code in your lab report.

Answer:



$$\text{Period of waveform (s)} = 2.0 \times 10^{-2}(\text{s})$$

$$\text{Pulse width, ON Time (s)} = 1.5 \times 10^{-3}(\text{s})$$

$$\text{Duty Cycle (\%)} = \frac{1.5 \times 10^{-3}}{2.0 \times 10^{-2}} \times 100\% = 7.5 (\%)$$

```

#include <avr/io.h>
#include "Arduino.h"

void setup() {

    cli();

    // 1. Set PB1 (Pin 9 on Arduino) as output
    DDRB |= (1 << PB1);

    // For falling edge on INT0, Bits 1 and 0 should be 10
    EICRA |= 0b00000011; // ADDED FOR PART 2

    // 2. Set Phase Correct PWM mode
    TCCR1A = 0b00000010;

    TIMSK1 = 0b10; // ADDED FOR PART 2

    // Set Prescaler
    // This has to be set early for Timer1.
    // By default, the Arduino Wiring language already
    // sets up Timer1 in a different mode before this, which
    // makes our OCR1A = .. result in a wrong value otherwise
    TCCR1B = 0b00010010; //prescaler is 8

    // 3. Set ICR1
    ICR1 = 20000;

    // 4. Set OCR1A for duty cycle
    OCR1A = 1500; // 0° for servo

    TCNT1 = 0;

    // 5. Enable Non-Inverting Mode on OC1A (PB1)
    TCCR1A |= (1 << COM1A1);

    sei();
}

void loop() {

}

```

Q10: [4 marks]

The servo motor can rotate from 0° to 180°. Modify your code to make the servo continuously sweep from 0° to 180° and back to 0° in a smooth motion. Ensure a gradual transition by adjusting the pulse width incrementally. You are allowed to use the delay function for smoothness, but not to actually generate the PWM pulses themselves. a) Include your code in your lab report. b) Show your servo motor behavior to your TA.

Answer:

```
#define SERVO_SPEED 20 // Varies the speed of flashing cycle of LED
static volatile bool counting_up = false; // Variable that toggles between modes

ISR(TIMER1_COMPA_vect) {
    if (OCR1A <= 500) { // minimum angle (-90°) for servo
        counting_up = true;
    } else if (OCR1A >= 2800) { // max angle (+90°) for servo
        counting_up = false;
    }

    if (counting_up) {
        OCR1A += SERVO_SPEED;
    } else {
        OCR1A -= SERVO_SPEED;
    }
}
```